Architectural Tradeoffs in Low-Latency Algorithmic Trading with Predictive Signals

Abhinav Randive

March 5, 2026

1 Introduction

Financial markets have increasingly become dominated by automated trading systems that make decisions and execute trades at extremely high speeds. These systems rely on algorithmic strategies that analyze market data and react within milliseconds or even microseconds. As a result, the design of trading infrastructure has become just as important as the predictive models used to generate trading signals. Even when predictive models are accurate, delays in processing or order execution can eliminate any potential advantage in highly competitive markets.

Recent advances in machine learning have led to significant interest in applying predictive models to financial markets. Researchers have explored techniques ranging from classical statistical models to deep learning architectures for forecasting stock price movements. These models often attempt to identify patterns in historical price data, macroeconomic indicators, or market sentiment signals in order to generate short-term predictions about market behavior.

However, predictive accuracy alone does not guarantee profitability in financial markets. In practice, the ability to act on predictions depends on the latency of the trading system and the structure of the market itself. Modern electronic markets operate using limit order books and price-time priority mechanisms, meaning that orders arriving even slightly later than competitors may lose execution priority. This creates a critical interaction between prediction, system architecture, and market microstructure.

The goal of this project is to study these architectural tradeoffs by developing an event-driven trading simulation system. The system replays historical market data, processes events sequentially, generates short-term predictive signals, and measures the impact of latency and execution constraints on simulated trading outcomes. By focusing on the infrastructure that connects predictive models with market execution, this work aims to bridge the gap between machine learning research and practical trading system design.

In addition to predictive modeling challenges, the operational environment of financial markets introduces additional constraints. Modern exchanges operate at extremely high speeds and rely on complex technological infrastructures that process millions of messages per second. Trading systems must be designed to handle high-frequency streams of constant market data while maintaining a deterministic order and minimizing latency.

These constraints mean that the architecture of the trading system itself plays a crucial role in determining whether predictive signals can actually be translated into profitable trades. Even small delays in processing market data or submitting orders can result in lost queue position, reduced fill rates, and increased exposure to adverse price movements. Moreover, understanding the interaction between predictive models and trading system architecture has become an important area of study in quantitative finance and financial engineering.

2 Background

Understanding algorithmic trading systems requires familiarity with the structure of modern financial markets. Most electronic exchanges operate using a limit order book, where buy and sell orders are organized by price and time of submission. When traders submit limit orders, they specify a price at which they are willing to buy or sell an asset. These orders are placed into a queue and executed when matching orders arrive on the opposite side of the market.

The concept of price-time priority plays a central role in these systems. Orders with better prices are executed first, but when multiple orders exist at the same price level, they are executed in the order they were received. This means that execution speed can significantly affect trading outcomes. Even small delays in order submission can cause a trader to lose queue position, reducing the likelihood of execution. Harris provides a detailed overview of these market microstructure mechanisms and their implications for trading strategies [5].

HFT trading systems are specifically designed to minimize such delays. According to Aldridge, high-frequency trading firms routinely invest heavily in low-latency infrastructure, including using specialized hardware, software, and proximity hosting, trying to be as close to the exchange servers as possible [1]. These trading systems monitor and process incoming market data and update trading decisions in real time in microseconds. The performance of such systems depends not only on the accuracy of prediction models but also on the efficiency of event processing, memory access, and network communication.

Cartea et al. [8] further emphasizes that successful algorithmic trading strategies must always account for both predictive signaling (BUY/SELL/HOLD) and market impact. Trades can influence market prices, and poorly designed strategies may result from terrible selection, where informed traders exploit predictable behavior. As a result, realistic simulation environments must inlcude all aspects of market microstructure when evaluating a given trading algorithm.

3 Related Work

Research on algorithmic trading spans several disciplines, including machine learning, financial economics, and computer systems. Prior work in this area can broadly be divided into three categories: machine learning approaches for market prediction, studies of market microstructure, and research on real-time systems architecture.

3.1 Machine Learning for Financial Prediction

A large body of research has explored the use of machine learning techniques for predicting financial market behavior. Early approaches relied primarily on linear regression models and statistical methods, but recent work has increasingly focused on deep learning architectures capable of capturing complex temporal patterns in financial data.

Fischer and Krauss demonstrate that Long Short-Term Memory (LSTM) neural networks can outperform traditional machine learning models when predicting stock market returns using historical price data [3]. Their results highlight the ability of deep learning models to capture long-term dependencies in financial time series that may not be easily identified by classical statistical techniques.

Similarly, Gu, Kelly, and Xiu conduct a comprehensive empirical study of machine learning methods applied to asset pricing [4]. Their research evaluates a wide range of algorithms, including neural networks and tree-based models, using a large set of financial predictors. The authors find that machine learning approaches can improve predictive performance relative to traditional linear models, particularly when working with high-dimensional datasets.

Despite these advances, predictive modeling in financial markets remains challenging due to the non-stationary nature of financial time series. Statistical relationships between variables can change over time as market conditions evolve, meaning that models which perform well during historical backtesting may degrade when deployed in live trading environments. Consequently, predictive accuracy alone is often insufficient for evaluating the real-world effectiveness of trading strategies.

3.2 Market Microstructure and Trading Dynamics

Another important area of research focuses on market microstructure, which studies the mechanisms through which trades occur in financial markets. Modern electronic exchanges operate using limit order books, where buy and sell orders are matched according to price-time priority.

Bouchaud, Farmer, and Lillo analyze how markets absorb changes in supply and demand over time [2]. Their work demonstrates that price dynamics emerge from the interaction of orders within the limit order book rather than purely from external information shocks. This perspective highlights the importance of understanding liquidity and order flow when designing algorithmic trading strategies.

Recent research has also explored the integration of alternative data sources into predictive models. Patsiarikas et al. [6] investigate the use of macroeconomic indicators, technical signals, and sentiment data for stock market forecasting. Their findings suggest that combining multiple sources of information can improve predictive performance compared to models that rely solely on historical price data.

However, most studies in this area focus primarily on improving forecast accuracy rather than examining the operational constraints involved in executing trading strategies within real market environments.

3.3 Systems Architecture and Low-Latency Processing

Beyond predictive modeling and market microstructure, the design of trading system infrastructure has also been studied within the computer systems literature. Modern financial markets generate extremely large volumes of real-time data, requiring trading platforms to process continuous streams of market events with minimal latency.

Stonebraker et al. [7] describe several key requirements for real-time stream processing systems, including deterministic event ordering, efficient memory usage, and the ability to process high-frequency data streams without introducing significant processing delays. These principles are directly applicable to algorithmic trading systems.

Event-driven architectures are commonly used in trading systems because they allow market activity to be represented as sequences of discrete events that can be processed in strict chronological order. Such systems must ingest market data, generate trading decisions, and submit orders within extremely short time windows.

Despite the importance of these architectural considerations, relatively few studies examine the interaction between predictive models and the software infrastructure used to execute trading strategies. Many machine learning studies assume idealized trading conditions in which predictions can be executed instantaneously.

4 Methodology

This project begins by constructing the foundational infrastructure required for an event-driven trading system simulation. At this stage, the focus is on market data ingestion and deterministic event replay, which form the basis for analyzing architectural tradeoffs in low-latency trading systems.

4.1 Market Data Acquisition

Historical market data is downloaded using the Yahoo Finance API through the `yfinance` Python library. The current implementation retrieves recent S&P 500 index data at one-minute intervals and stores timestamped price and volume information in CSV format. The preprocessing step flattens multi-index columns, standardizes column names, and converts timestamps to integer Unix time for consistent processing.

This approach ensures reproducibility and establishes a structured input format for the simulation pipeline. Using historical replay rather than live feeds aligns with common practices in quantitative finance research, where controlled backtesting environments are used to evaluate trading logic [1, 8].

4.2 Event Abstraction

To support an event-driven architecture, market data is abstracted into discrete events. An `Event` data structure is defined with three fields: timestamp, event type, and payload. Event types are enumerated to allow future extension for order submission, cancellation, and execution events.

Each market update is represented as a structured event object, enabling uniform handling of data within the system. This abstraction follows principles of real-time stream processing systems, where events are processed sequentially and deterministically [7]. Event abstraction is critical for modeling realistic trading environments, where exchanges operate as event-driven systems governed by limit order book mechanics [5].

4.3 Deterministic Market Replay

A replay engine iterates sequentially over historical market data and converts each row into a `MARKET_UPDATE` event. The replay mechanism maintains an internal index and exposes methods to check for remaining events and retrieve the next event in chronological order.

Deterministic replay prevents look-ahead bias and ensures that events are processed strictly in historical sequence. This is essential when studying the interaction between prediction and execution, since improper ordering can artificially inflate model performance [2]. Although a full event dispatcher and execution engine have not yet been implemented, the replay system establishes the primary input stream for the future low-latency architecture.

5 Conclusion

References

[1] Aldridge, I. *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*, 2nd ed. John Wiley & Sons, 2013.

[2] Bouchaud, J.-P., Farmer, J. D., and Lillo, F. How markets slowly digest changes in supply and demand. *Handbook of Financial Markets: Dynamics and Evolution* (2009), 57–160.

[3] Fischer, T., and Krauss, C. Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research 270*, 2 (2018), 654–669.

[4] Gu, S., Kelly, B., and Xiu, D. Empirical asset pricing via machine learning. *The Review of Financial Studies 33*, 5 (2020), 2223–2273.

[5] Harris, L. *Trading and Exchanges: Market Microstructure for Practitioners.* Oxford University Press, 2003.

[6] Patsiarikas, M., Papageorgiou, G., and Tjortjis, C. Using machine learning on macroeconomic, technical, and sentiment indicators for stock market forecasting. *Information 16*, 2 (2025).

[7] Stonebraker, M., Çetintemel, U., and Zdonik, S. The 8 requirements of real-time stream processing. *Association for Computing Machinery Special Interest Group on Management of Data Record 34*, 4 (2005), 42–47.

[8] Álvaro Cartea, Jaimungal, S., and Penalva, J. *Algorithmic and High-Frequency Trading*. Cambridge University Press, 2015.